

Indian Institute of Technology Jammu
CT1 — Computer Systems and Programming (CS-1-01 MO)
SOLUTIONS — every question with its answer and reasoning

Time: 1 Hour	Maximum Marks: 50
Name: _____	Roll No.: _____

Instructions:

1. Attempt as many questions as you like — there is no penalty for extra attempts.
2. Section A: your **best 5 of 7** count (2 marks each).
3. Section B: your **best 5 of 7** count (4 marks each).
4. Section C: your **best 2 of 3** count (10 marks each).
5. Show your working. Answers without steps earn partial credit at best.

SECTION A — Multiple Choice Questions

Best 5 of 7. $5 \times 2 = 10$ Marks

1. Which decimal number is $(14)_{16}$?
A. 14 B. 18 C. 20 D. 41

Key: C — $1 \times 16 + 4 = 20$; the digits “14” do not mean fourteen — meaning depends on the base.

2. In 4-bit **one’s complement**, how many distinct bit patterns represent the value zero?
A. 0 B. 1 C. 2 D. 16

Key: C — 0000 (+0) and 1111 (−0): the famous flaw, at any width.

3. What is the chief advantage of **two’s complement** over sign-magnitude and one’s complement?
A. It can represent more positive numbers
B. Zero has exactly one pattern, and one ordinary binary adder handles all signs
C. Negative numbers need fewer bits
D. Overflow can never happen

Key: B — no −0, no special cases: subtraction becomes addition of a complement on the same hardware. (A is false: it gains one extra *negative* value, not positive.)

4. A 4-bit two’s-complement machine computes $5 + 6$. The stored result represents:
A. 11 B. −5 C. −6 D. 1

Key: B — 11 does not fit in $-8 \dots 7$; pattern 1011 reads as $11 - 16 = -5$. Overflow.

5. You run `gcc hello.c -o hello`. What is inside the new file `hello` that was *not* inside `hello.c`?
A. A neater, error-checked copy of the C text
B. Machine instructions the processor can execute directly
C. The output “Hello World”, computed in advance
D. Nothing new — it is the same file under another name

Key: B — the compiler *translates* the plan from C into the processor’s own instruction codes; `./hello` hands those to the processor. Nothing has run yet, so C is impossible.

6. What exactly does this program print?

```
#include <stdio.h>
int main(void) {
    printf("Hello");
    printf(" World\n");
    printf("CS-1-01");
    return 0;
}
```

- A. Hello, World, CS-1-01 on three separate lines
- B. Hello World on one line, then CS-1-01 on the next
- C. Hello World CS-1-01 all on one line
- D. HelloWorld\nCS-1-01 printed literally

Key: B — `printf` never adds newlines on its own; only the one `\n` breaks the line.

7. According to this course, programming is ultimately about:

- A. memorizing the syntax of C correctly
- B. making the computer run as fast as possible
- C. solving a problem — turning a precise plan into instructions an executor can carry out, then checking the result
- D. writing as many lines of code as possible

Key: C — the computer is the instrument; the plan and its verification are the subject.

SECTION B — Short Answer Questions

Best 5 of 7. $5 \times 4 = 20$ Marks

1. Two conversions, showing every step:

- (a) Convert $(11001)_2$ to decimal, writing out the powers of 2 you used.
- (b) Convert $(25)_{10}$ to hexadecimal, using repeated division by 16.

Key: (a) $16 + 8 + 1 = 25$. (b) $25 = 1 \times 16 + 9 \rightarrow 19_{16}$. (Yes: the same twenty-five is written “19” in base 16 — digits only mean something relative to the base.)

2. Write -5 in 4 bits under each convention, showing your steps:

- (a) sign-magnitude
- (b) one’s complement
- (c) two’s complement

Key: $+5 = 0101$. (a) 1101 (b) 1010 (c) 1011.

3. On a 4-bit **one’s complement** machine, add $(+3)$ and (-3) by binary addition, showing every bit. What pattern do you obtain, what value does it represent, and what does this tell you about why two’s complement was preferred?

Key: $0011 + 1100 = 1111 = -0$. A “second zero”: equality and hardware need special cases; two’s complement has one zero and one adder for everything.

4. On a 4-bit two's-complement machine, compute $3 - 5$ the way the hardware does: form the two's complement of 5, add it to 3 showing every bit (discard any carry out of the fourth bit), and interpret the resulting pattern. What circuit did the machine just avoid needing?

Key: $5 = 0101 \rightarrow$ two's complement 1011 ; $0011 + 1011 = 1110 = -2$. ✓ Subtraction ran on the ordinary adder — no separate subtractor circuit needed. This is the two's-complement advantage in action.

5. (a) Take the 4-bit pattern $d = 0110$. Compute the one's complement of its one's complement, showing both steps. State what you observe, and argue in one sentence why it must hold for *every* pattern. [2]

(b) For a k -bit machine, the two's complement of d can be computed as the unsigned number $2^k - d$. Write the analogous formula for the *one's* complement, and verify your formula for $d = 5$, $k = 4$ against bit-flipping. [2]

Key: (a) $0110 \rightarrow 1001 \rightarrow 0110$: back where we started; each bit flipped twice is itself (an involution). (b) $(2^k - 1) - d$: all-ones minus d flips every bit. Check: $15 - 5 = 10 = 1010$, which is 0101 flipped. ✓

6. Mark each statement **True or False** and justify with a concrete example:

(a) “Every real number can be represented exactly on a computer.”

(b) “No real number can be represented exactly on a computer.”

Key: (a) **False:** a k -bit format has at most 2^k patterns — finitely many values — while the reals are infinite; e.g. π (or 0.1, an infinite repeating binary fraction) is stored rounded. (b) **Also false:** any value of the form $m \times 2^e$ within range is exact — e.g. $0.5 = 2^{-1}$, or $6.5 = 1.101 \times 2^2$, stored exactly in IEEE format. Full marks need both verdicts and one example each.

7. A famous line — which the internet confidently attributes to Edsger Dijkstra — says that computer science is about computers as much as astronomy is about telescopes.

(a) What is the analogy claiming? If programming is not “about” the computer, what *is* it about? Support your answer with one concrete example from this course (the Turtle, the Toy Computer, or your first C program). [2]

(b) The attribution above is, in fact, wrong — and that is itself a lesson this course keeps repeating. In one sentence: what should you do with a claim that is plausible and widely repeated? [2]

Key: (a) The computer is the *instrument*, not the subject — the subject is computing: plans (algorithms) made precise enough to be executed and checked. Example: the Turtle — no silicon in sight, yet everything that mattered was the plan; or the same add-two-numbers plan surviving unchanged from toy words to C. (b) Verify it against a source — plausible and widely repeated is not the same as true (the verified origin is Michael Fellows with Ian Parberry, early 1990s; accept any answer expressing “check before believing”).

SECTION C — Long Answer Questions

Best 2 of 3. $2 \times 10 = 20$ Marks

1. All in 8-bit two's complement, showing every carry:

(a) Compute $93 + 45$ in binary. State the decimal value the stored pattern represents, whether overflow occurred, and *how you can tell without converting back*. [5]

(b) Compute $93 - 45$ *as an addition* (using the two's complement of 45), showing the steps and what happens to the ninth bit. [5]

Key: (a) $01011101 + 00101101 = 10001010 = -118$; overflow — two positives produced a “negative” sign bit. (b) $-45 = 11010011$; $01011101 + 11010011 = 1\ 00110000$; ninth bit falls off the 8-bit edge; result $00110000 = 48$. ✓

2. For a k -bit two’s-complement machine:

(a) What is the **largest positive** integer it can represent, and what is the **most negative**? Justify each in a sentence or two — think about how the sign bit splits the 2^k available patterns — and explain why the two ends are not symmetric. [4]

(b) Now let the machine test your claim. On a **4-bit** machine your answer says the largest is $7 = 0111$. Add 1 to it in binary. What pattern appears, what value does it represent, and does the result support or contradict your claim in (a)? What is this phenomenon called? [3]

(c) Repeat the test at the other end: take the most negative value and add $-1 = 1111$, showing what happens to the carry. What do you observe, and what is *this* called? [3]

Key: (a) Largest $2^{k-1} - 1$ (0 then all ones); most negative -2^{k-1} (1 then all zeros). Half the patterns (2^{k-1}) carry the sign bit and serve the negatives; the other half must cover zero *and* the positives, so the positives get one fewer. (b) $0111 + 0001 = 1000 = -8$: one past the largest lands on the most negative. This *supports* the claim — there is nowhere higher to go, so the count wraps around. Overflow. (c) $1000 + 1111 = 1\ 0111$; the fifth bit falls off the 4-bit edge, leaving $0111 = +7$: one past the most negative wraps to the largest positive. Underflow. The two tests bracket the range exactly where (a) predicted. *Also accepted:* most architecture texts call this *negative overflow*, reserving “underflow” for floating-point values too small to represent. Either term earns full marks.

3. A toy machine has a single box for a number, and it obeys — blindly, one line at a time, top to bottom — exactly these four instructions:

```
LOAD n    put the number n into the box
ADD n     add n to what the box holds
DOUBLE    double what the box holds
PRINT     print the box's value on its own line
```

Consider this program:

```
LOAD 3
ADD 2
DOUBLE
PRINT
ADD 1
PRINT
```

(a) Predict the exact output, tracing the box’s value step by step. [3]

(b) The lines ADD 2 and DOUBLE are now swapped. Predict the new output. [3]

(c) The machine did not change between (a) and (b), yet the output did. In one sentence: where does the “meaning” of a program live? [2]

(d) Using only these four instructions, write a program that prints exactly 7 and then 14, on two lines. [2]

Key: (a) 10 then 11 — box: $3 \rightarrow 5 \rightarrow 10$ (printed), $\rightarrow 11$ (printed). (b) 8 then 9 — box: $3 \rightarrow 6 \rightarrow 8$ (printed), $\rightarrow 9$ (printed). (c) In the plan — the order of the instructions — not in the machine, which only executes one step at a time without understanding. (d) e.g. LOAD 7 / PRINT / DOUBLE / PRINT; any correct program earns full marks — shorter is more elegant, not more correct.

— End of Question Paper —